



Smart Review Analyzer

A Complete NLP Sentiment Analysis Pipeline



Team Members

Salma Mohammed

Sameh Naeem

Rawan Essam

Mohammed Saied

Malak Tarek

Ahmed Khaled



What Problem Are We Solving?

- Huge amount of user reviews online
- Hard to manually analyze them
- Businesses need quick insights

Companies receive thousands of reviews, and manually reading them is inefficient. We built a system that automatically analyzes sentiment and extracts insights.



Objectives

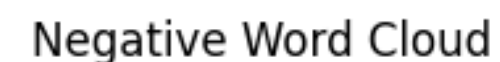
- 1 Classify reviews → Positive / Negative
- 2 Extract meaningful insights
- 3 Identify patterns in user feedback
- 4 Provide statistics & visualization



2

- great
- love
- best

- bad
- movie
- worst



System Pipeline

1. Raw Reviews (IMDB Dataset)

2. Text Preprocessing

- Lowercase
- Stopwords Removal
- Lemmatization

3. Feature Extraction

- TF-IDF
- Word2Vec

4. Model Training

- Logistic Regression
- LSTM
- BERT

5. Evaluation

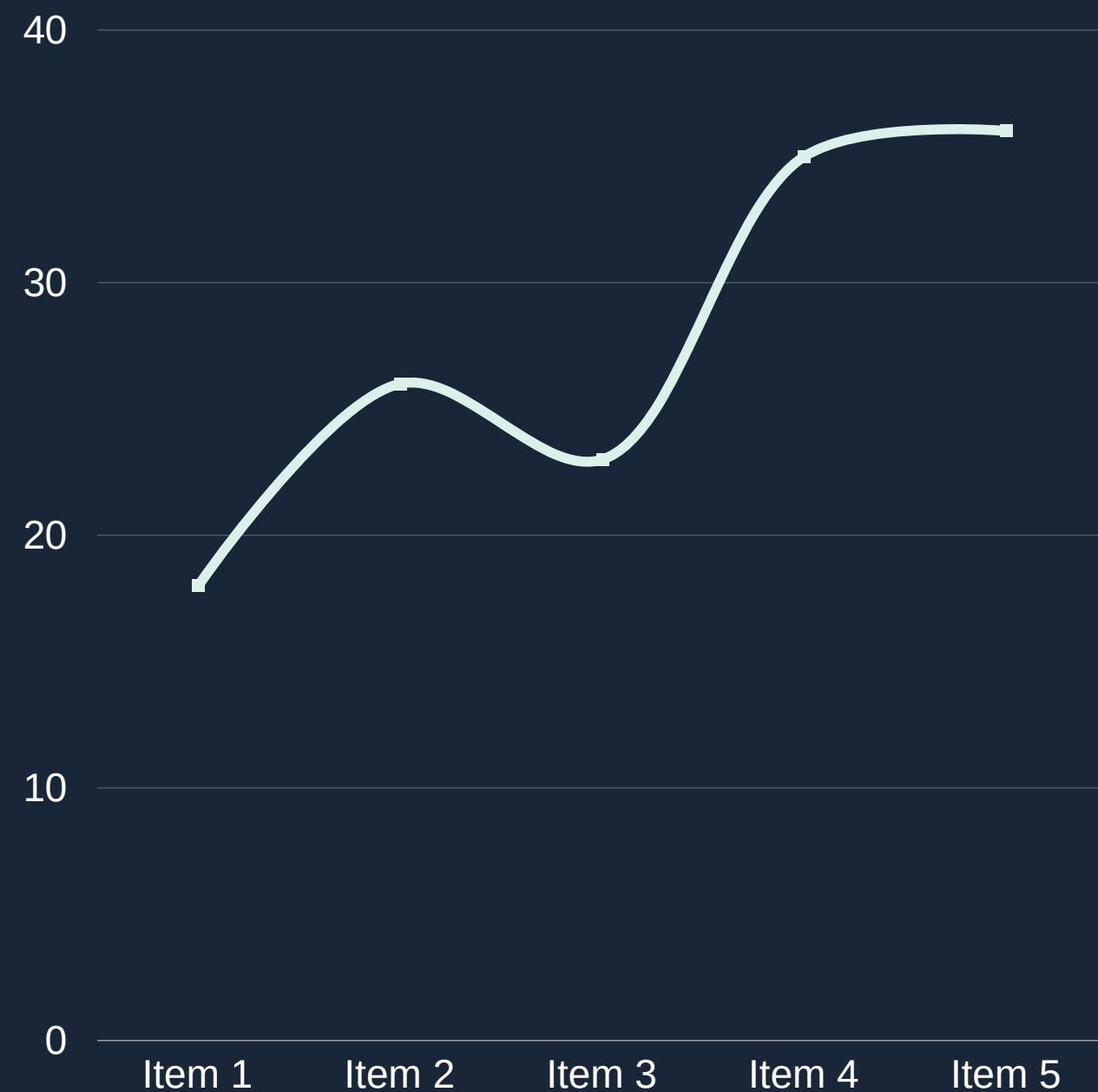
- Accuracy
- Precision
- Recall
- F1-score

6. Insights & Visualization

- WordCloud
- Confusion Matrix
- Charts



Dataset



- Column names:
 - `Index(['review', 'sentiment'], dtype='object')`
- Number of rows: 50000
- Number of columns: 2
- Missing values:
 - review: 0
 - sentiment: 0
- Class distribution:
 - sentiment:
 - positive: 25000
 - negative: 25000



Preprocessing Steps

1. Convert Text to Lowercase
2. Remove Punctuation
3. Remove Stopwords
4. Tokenization
5. Lemmatization
6. Text Normalization

Tokenization

```
In [58]: df['tokens'] = df['cleaned_review'].apply(lambda x: x.split())
```

```
In [59]: print(df[['cleaned_review', 'tokens']].head(3))
```

	cleaned_review \	tokens
0	one reviewer mentioned watching 1 oz episode y...	[one, reviewer, mentioned, watching, 1, oz, ep...
1	wonderful little production filming technique ...	[wonderful, little, production, filming, techn...
2	thought wonderful way spend time hot summer we...	[thought, wonderful, way, spend, time, hot, su...



Feature Extraction

1. TF-IDF (Term Frequency-Inverse Document Frequency)

- Converts text into numerical vectors
- Measures importance of words in each review
- Used for Logistic Regression model
- Vocabulary size: 5000 words

2. Word2Vec Embeddings

- Converts words into dense vectors
- Captures semantic relationships between words
- Used for LSTM model
- Vector size: 100 dimensions



Model Pipeline

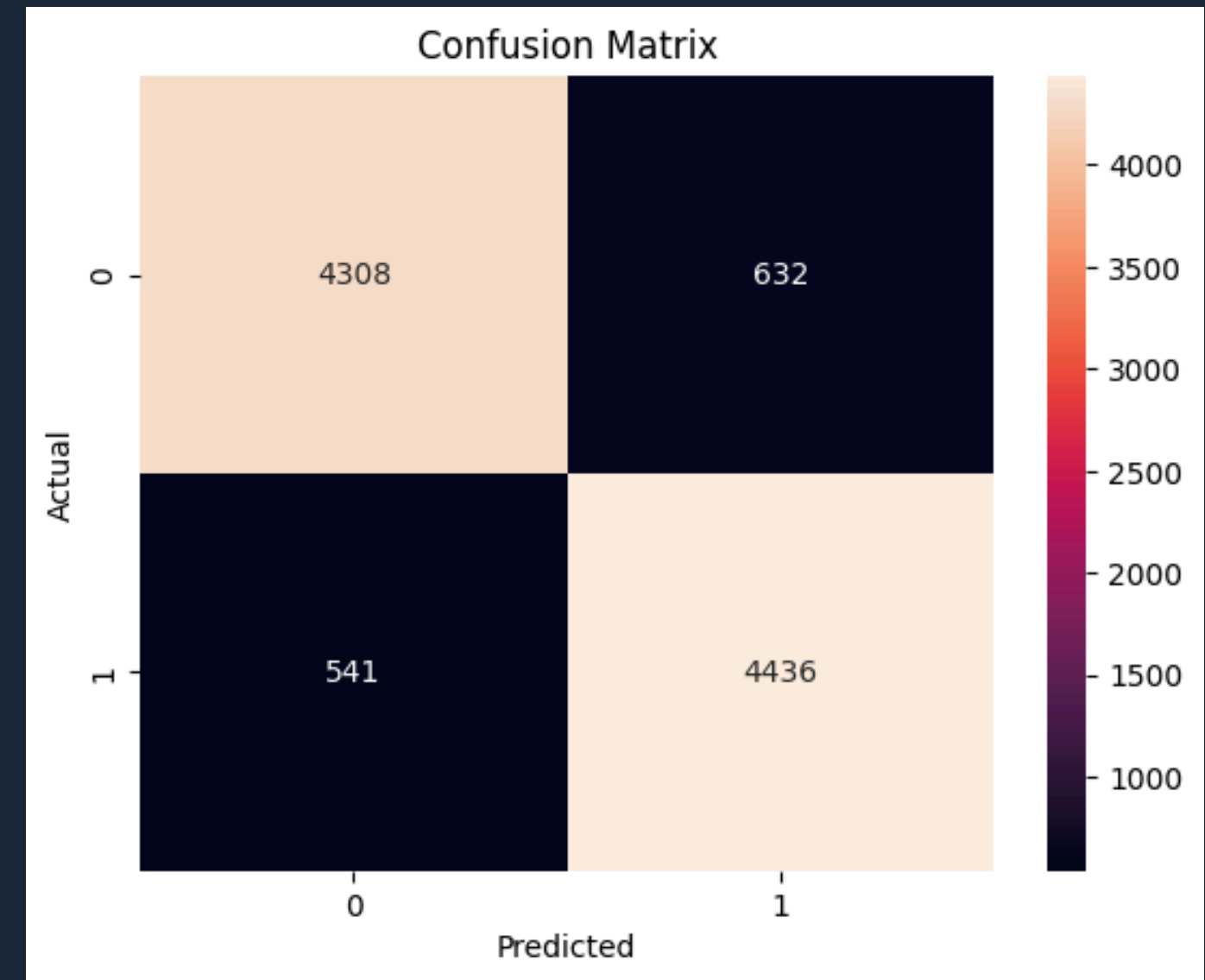
- 1 Baseline → Logistic Regression
- 2 Intermediate → LSTM
- 3 Advanced → BERT



Baseline Model

Logestic Regression :

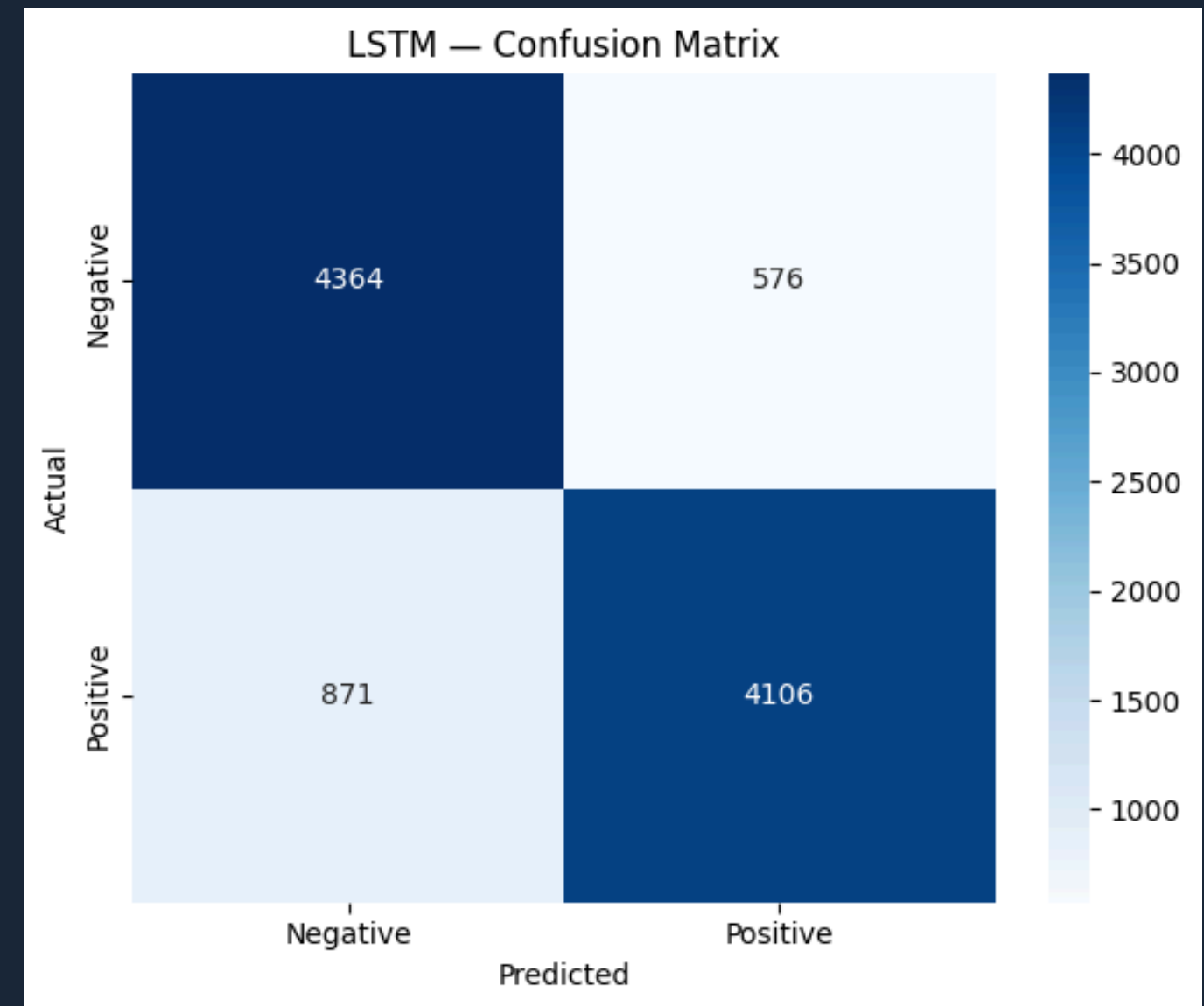
- Uses TF-IDF features
- Fast training time
- Simple and efficient
- Strong baseline performance
- Results:
 - Accuracy: 88%
 - Precision: 0.88
 - Recall: 0.88
 - F1-score: 0.88



Intermediate Model

Deep Learning Model — LSTM :

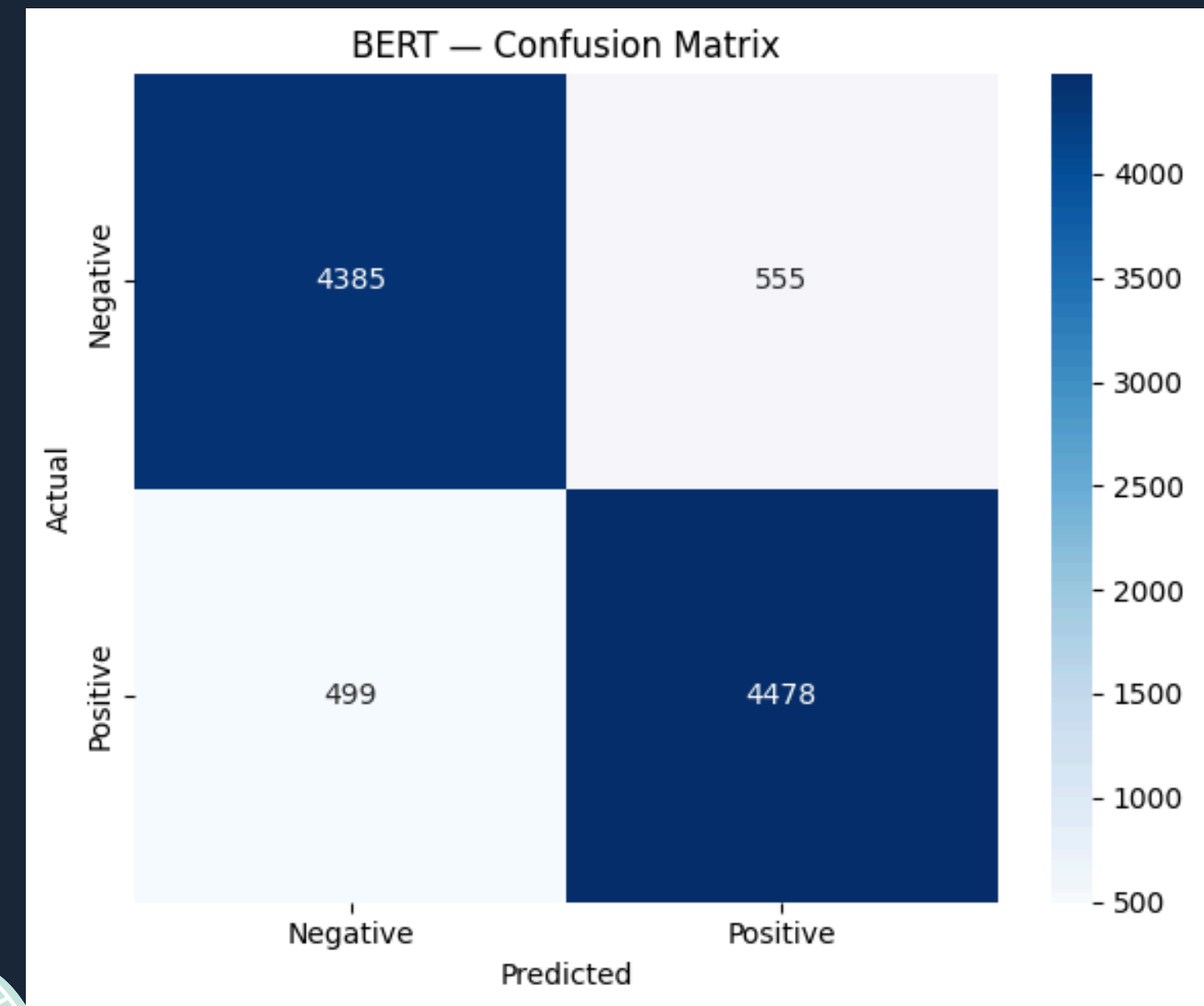
- Uses Word2Vec embeddings
- Captures sequence relationships
- Understands context in text
- Handles long-term dependencies
- Results:
 - Accuracy: 85.41%
 - Precision: 0.8770
 - Recall: 0.8250
 - F1-score: 0.8502



Advanced Model

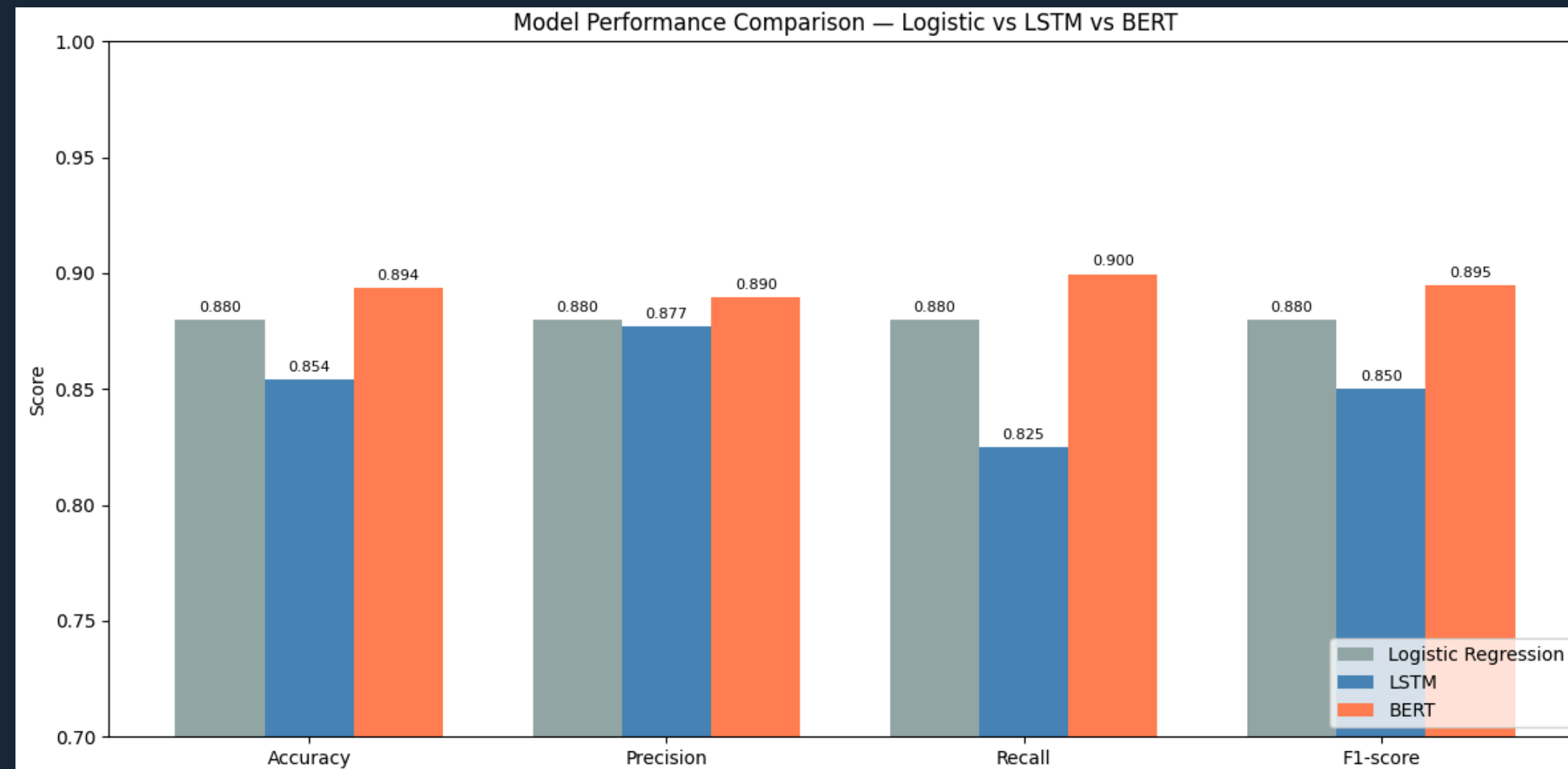
Transformer Model — BERT:

- Pretrained transformer model
- Understands bidirectional context
- Fine-tuned on our dataset
- Achieved the best performance
- Results:
 - Accuracy: 89.37%
 - Precision: 0.8897
 - Recall: 0.8997
 - F1-score: 0.8947



Comparison

- BERT is the Best Performing Model



Comparison

- BERT is the Best Performing Model

Model	Accuracy	Precision	Recall	F1-score
Logistic Regress	0.88	0.88	0.88	0.88
LSTM	0.8541	0.877	0.825	0.8502
BERT	0.8937	0.8897	0.8997	0.8947



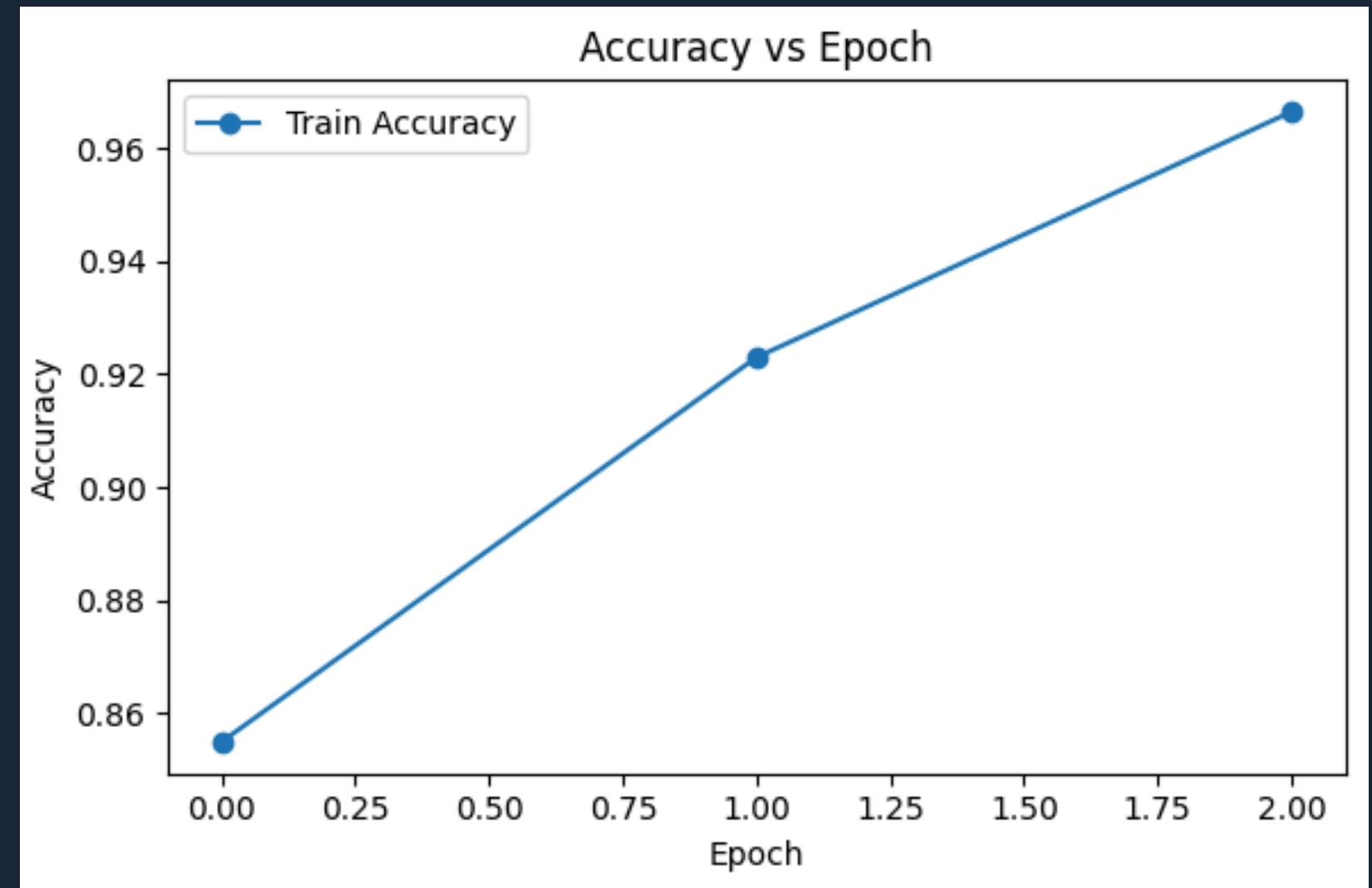
Evaluation Metrics

- 1 Accuracy:
 - Measures overall correctness
- 2 Precision
 - Measures correctness of positive predictions
- 3 Recall
 - Measures how many positives were detected
- 4 F1-score
 - Balance between Precision and Recall
- 5 Confusion Matrix
 - Shows prediction errors



Findings

- **BERT** achieved the best performance
- **Logistic Regression** provided fast results
- **LSTM** captured sequence relationships
- **TF-IDF** and **Word2Vec** improved model learning
- Multiple models improved evaluation reliability



Conclusion

- Built a complete NLP pipeline
- Applied preprocessing and feature extraction
- Trained and compared three models
- Generated visual insights
- Achieved high classification performance





Thank You

by NLP Team

